

Servo Controller mit Rotary Encoder & LCD (Raspberry Pi)

- Von Jannik Seitz & Jan Wilhelm -

Hier geht's zu unserem GitHub-Repository: https://github.com/ThePhoenixPixel/raspberry_lcd_motor

Projektdokumentation

1. Ziel des Projekts

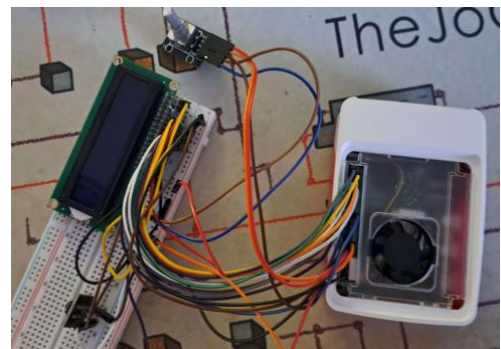
Ziel des Projekts war es, einen Servo-Motor mithilfe eines Rotary Encoders zu steuern. Zusätzlich sollten zwei Positionen gespeichert und über einen Taster abgerufen werden. Die aktuelle Einstellung sowie ein Menü werden über ein 16×2 LCD-Display angezeigt.

Der Fokus lag dabei sowohl auf dem praktischen Aufbau der Schaltung als auch auf der Umsetzung der Steuerlogik in Python.

2. Praktischer Aufbau

2.1 - Verwendete Komponenten

- Raspberry Pi
- Rotary Encoder (KY-040)
- 16×2 LCD
- Zwei Taster
- Servo-Motor



2.2 - Verkabelung

Rotary Encoder

Der Encoder wurde direkt an digitale GPIO-Pins angeschlossen:

- VCC → 5V
- GND → GND
- CLK → GPIO17 (BCM)
- DT → GPIO27 (BCM)

Die Signale des Encoders sind digital, daher sind keine analogen Eingänge erforderlich.

LCD-Display

Das LCD wurde im 8-Bit-Modus betrieben und über das BOARD-Nummerierungssystem angeschlossen:

- RS → Pin 37
- E → Pin 35
- D0–D7 → Pins 40, 38, 36, 32, 33, 31, 29, 23

Taster

- Menü-Taster → Pin 16
- Toggle-Taster → Pin 22

Beide Taster wurden mit Pull-Up-Widerständen konfiguriert, sodass sie beim Drücken ein LOW-Signal liefern.

Servo

- Signal → Pin 12 (PWM)
- Versorgung → externe 5V (empfohlen)
- Masse → gemeinsame Masse mit Raspberry Pi

3. Software – Grundstruktur

Das Programm wurde in Python umgesetzt und nutzt folgende Bibliotheken:

- RPi.GPIO für GPIO und PWM
- gpiozero für den Rotary Encoder
- RPLCD für das LCD

Zu Beginn wird der GPIO-Modus festgelegt:

```
GPIO.setmode(GPIO.BOARD)
```

Damit beziehen sich alle Pinangaben (außer beim Encoder) auf die physischen Pins.

4. Initialisierung der Hardware im Code

LCD

Das Display wird über die Klasse CharLCD initialisiert:

```
lcd = CharLCD(  
    cols=16,  
    rows=2,  
    pin_rs=37,  
    pin_e=35,  
    pins_data=[40, 38, 36, 32, 33, 31, 29, 23],  
    numbering_mode=GPIO.BOARD  
)
```

Hier werden alle verwendeten Pins sowie die Displaygröße festgelegt.

Rotary Encoder

```
encoder = RotaryEncoder(a=17, b=27, max_steps=0)
```

Wichtig ist hier, dass die Pins im BCM-Modus angegeben werden.

Die Bibliothek übernimmt automatisch die Auswertung der Drehbewegung.

Taster und Servo

```
GPIO.setup(save_button, GPIO.IN, pull_up_down=GPIO.PUD_UP)  
GPIO.setup(toggle_button, GPIO.IN, pull_up_down=GPIO.PUD_UP)  
GPIO.setup(servo_pin, GPIO.OUT)
```

Die Taster werden als Eingänge mit Pull-Up definiert.

Der Servo wird über PWM gesteuert:

```
servo = GPIO.PWM(servo_pin, 50)  
servo.start(0)
```

5. Zentrale Programmvariablen

Die Steuerung basiert auf mehreren Zustandsvariablen:

- pos1, pos2: gespeicherte Winkel
- menu_active: gibt an, ob das Menü aktiv ist
- menu_selection: Auswahl im Menü (0 oder 1)
- current_toggle: merkt sich die letzte Position

Diese Variablen bestimmen das Verhalten des gesamten Programms.

6. Funktionsweise der Software

6.1 - Servo-Ansteuerung

Die Funktion zur Steuerung des Servos:

```
def set_servo_angle(angle):  
    angle = max(0, min(180, angle))  
    duty = 2 + (angle / 18)  
    servo.ChangeDutyCycle(duty)  
    time.sleep(0.5)  
    servo.ChangeDutyCycle(0)
```

- Begrenzung des Winkels auf 0 - 180 Grad
- Umrechnung in PWM-Signal
- kurzes Ansteuern des Servos
- Abschalten zur Vermeidung von Zittern

6.2 - LCD-Aktualisierung

Die Anzeige wird über eine zentrale Funktion gesteuert:

```
def update_lcd():  
    lcd.clear()
```

Je nach Zustand werden unterschiedliche Inhalte angezeigt:

- Normalmodus: aktueller Winkel
- Menümodus: Auswahl zwischen Pos1 und Pos2

6.3 - Verarbeitung der Drehbewegung

Die Drehbewegung wird über zwei Funktionen verarbeitet:

```
def rechts():  
def links():
```

Funktion im Normalmodus:

- Änderung des aktuellen Winkels

Funktion im Menümodus:

- Wechsel zwischen Pos1 und Pos2

Die Zuordnung erfolgt automatisch:

```
encoder.when_rotated_clockwise = rechts  
encoder.when_rotated_counter_clockwise = links
```

6.4 - Tastersteuerung

Im Hauptprogramm werden die Taster kontinuierlich abgefragt.

Menü-Taster

```
if GPIO.input(save_button) == GPIO.LOW:
```

- Aktiviert oder deaktiviert das Menü
- aktualisiert die Anzeige

Toggle-Taster

```
if GPIO.input(toggle_button) == GPIO.LOW:
```

- bewegt den Servo abwechselnd zu Pos1 und Pos2
- merkt sich die aktuelle Position über current_toggle

7. Hauptprogramm (Ablauf)

Das Programm läuft in einer Endlosschleife:

```
while True:
```

Dabei werden:

- Taster abgefragt
- Zustände geändert
- Anzeige aktualisiert

Eine kleine Pause (sleep) sorgt für einfache Entprellung.

8. Beenden des Programms

Beim Beenden wird folgendes ausgeführt:

```
servo.stop()  
lcd.clear()  
GPIO.cleanup()
```

Dadurch werden:

- PWM gestoppt
- Display gelöscht
- GPIO-Pins zurückgesetzt